

Analytics Programming with R

Week 4 - Chapters 13 - 16

Group Manipulation

- ▶ In Data Analysis, "Data Munging" takes up about 80% of the analysis effort.
- ▶ Data Munging is the manipulation of data
- ▶ Requires repeated operations on different sections of data (split-apply-combine)
 - ▶ We split the data into discrete sections based on some metric, apply a transformation of some kind to each section, and then combine all the sections together
- ▶ The `apply()` function in R is used for Data Munging tasks
- ▶ Apply Family of functions include:
 - ▶ `Tapply`
 - ▶ `Lapply`
 - ▶ `Mapply`

Group Manipulation

- ▶ Must be used on a matrix,
 - ▶ All of the elements must be of the same type whether they are character, numeric or logical.
 - ▶ If used on some other object, such as a data.frame, it will be converted to a matrix first.
- ▶ Apply() accepts 3 arguments
 - ▶ 1st argument is the object we are working with
 - ▶ 2nd argument is the margin to apply the function over
 - ▶ 1 = operate over the rows
 - ▶ 2 = operate over the columns
 - ▶ 3rd argument is the function we want to apply.
 - ▶ Any following arguments will be passed on to that function. Apply will iterate over each row (or column) of the matrix, treating them as individual inputs to the first argument of the specified function.

Group Manipulation

LAPPLY and SAPPLY

- ▶ lapply works by applying a function to each element of a list and returning the results as a list.
 - ▶ The “l” in lapply stands for list
- ▶ To return the result of lapply as a vector instead, use sapply
- ▶ It is exactly the same as lapply in every other way.
- ▶ A vector is technically a form of a list, lapply and sapply also take a vector as their input.

Group Manipulation

MAPPLY

- ▶ The `mapply()` function stands for 'multivariate' apply. Its purpose is to be able to vectorize arguments to a function that is not usually accepting vectors as arguments.
- ▶ `mapply()` applies a Function to Multiple List or multiple Vector Arguments.
- ▶ Most people use loops instead of the `mapply()` function

Group Manipulation

OTHER FUNCTIONS

- ▶ There are other function of the apply family
- ▶ Most have been superceded by functions in the plyer, dplyr and purr packages.
- ▶ Some of the other functions in the apply family include
 - ▶ Tapply
 - ▶ Rapply
 - ▶ Eapply
 - ▶ Vapply
 - ▶ by

Group Manipulation

PLYR Package

- ▶ The core of the package contains functions such as `ddply`, `llply`, and `ldply`
- ▶ All of these functions perform split-apply-combine functions on data.
 - ▶ `ddply` takes in a `data.frame` and outputs a `data.frame`
 - ▶ `llply` takes in a list and outputs a list
 - ▶ `ldply` takes in a list and outputs a `data.frame`

Group Manipulation

PLYR and DPLYR Packages

- ▶ plyr has useful helper functions
 - ▶ **each()** lets us supply multiple functions to a function like `aggregate()`
 - ▶ Drawback is that when using `each()`, additional arguments can no longer be supplied to the functions.
 - ▶ **ldata.frame()** creates a reference to a `data.frame` so that subsetting is faster and more memory efficient.

Group Manipulation

DPLYR Packages

- ▶ Dplyr is meant to be used with data frames
- ▶ Is quickly becoming the defacto in data munging
- ▶ Uses "grammar of data" to perform data munging
 - ▶ Each step is done by a singel function that represents a verb.
 - ▶ Select
 - ▶ Filter
 - ▶ Mutate
- ▶ Piping was popularized within the dplyr package by allowing one function to be piped to another in a chain.

Group Manipulation

DPLYR Packages

- ▶ An important feature of dplyr is its capability to work with data stored in a database in much the same way it works with data in data.frames.
 - ▶ PostgreSQL
 - ▶ MySQL
 - ▶ SQLite
 - ▶ MonetDB
 - ▶ Google Big Query
 - ▶ Spark DataFrames
- ▶ For standard computations, R code is translated into equivalent SQL code.
- ▶ For arbitrary R code that cannot be easily translated into SQL, dplyr (experimentally) chunks the data into memory and runs the computations independently.
 - ▶ This enables data munging and analysis on data that would otherwise not fit in memory

Iterations with PURR

PURR

- ▶ Iteration over elements of a list or vector is a repetitive task in R typically using loops
- ▶ purrr is designed to utilize the functional programming paradigm.
- ▶ Functional programming is when a function depends only on its input arguments.
 - ▶ This enables us to iterate over a list and apply a function to each element independently.
 - ▶ This is mainly aimed at operating on lists, though it can also be used to apply non-vectorized functions to vectors.
- ▶ The foundation of the Purrr package is the map() function which applies a function to each element in a list independently and returns the results in a list of the same length.

Group Manipulation

PURR

- ▶ Using map always results in a list as the returned result, which is highly generalized but not always desired
- ▶ Purrr offers a number of map functions that specify the type of expected result.
 - ▶ If the specified type cannot be returned, then the functions return an error.

Function	Output Type
<code>map</code>	list
<code>map_int</code>	integer
<code>map_dbl</code>	numeric
<code>map_chr</code>	character
<code>map_lgl</code>	logical
<code>map_df</code>	data.frame

Group Manipulation

PURR

- ▶ Iterating over a data frame is simple because data frames are technically lists.
- ▶ We learned `mapply` can take two arguments to corresponding elements of two lists. The purrr analog is `pmap`, with `map2` as a special case when the function takes exactly two arguments.
- ▶ Iterating over lists is easier with **purrr**.
- ▶ In addition to the speed **purrr** ensures the results returned are as the programmer expected and was designed to work with pipes, which further enhances the user experience.

Data Reshaping

RESHAPING YOUR DATA

Cbind() and Rbind()

- ▶ Sometimes you will need to reshape your data
 - ▶ Column oriented to Row Oriented and vice versa.
- ▶ Two functions in R help do that
 - ▶ Cbind() – takes multiple vectors and combines them as columns in a data frame
 - ▶ Rbind() – takes multiple data frames and combines them into 1 data frame

Data Reshaping

RESHAPING YOUR DATA

Merge

Merge() is a built in function in R

Merge() – merges two data frames

- ▶ by.x specifies the left column of the data frame
- ▶ by.y specifies the right column of the data frame
- ▶ Merge() is limited to only 2 data frames
- ▶ Join() – found in the PLYR package
- ▶ Data.Table()

Data Reshaping

RESHAPING YOUR DATA

PLYR Join

- ▶ The `Join()` function works similar to `Merge()`
- ▶ `Join()` is faster than `merge()`
- ▶ Each key column in each table must have the same name
- ▶ There are two arguments in `join()` that specify various types of joins
 - ▶ Left join
 - ▶ Right join
 - ▶ Inner Join
 - ▶ Full (outer) join

Data Reshaping

RESHAPING YOUR DATA

Reduce() Function

- ▶ `reduce()` is an operation that combines the elements of a vector into a single value.
- ▶ Rather than using a loop to join multiple data frames together we use `Reduce` to do it quickly.

Data Reshaping

RESHAPING YOUR DATA

Data.table Merge

- ▶ Start by converting data frames into data.tables by using the `data.table()` function
- ▶ The join requires specifying the keys for each table

Data Reshaping

RESHAPING YOUR DATA

TIDYVERSE

- ▶ Tidyverse contains newer packages for data reshaping
 - ▶ Dplyr and tidyr
- ▶ Designed to work with pipes
- ▶ Speeds up performance in most cases
- ▶ Uses `bind_rows` to combine rows
- ▶ Uses `bind_cols` to combine columns
- ▶ ONLY WORKS WITH DATA FRAMES or TIBBLES